LAB3

Lab3

- The deadline for lab3 submission is 19th March at 11:59 pm.
- If you have any questions, please contact TA through email. (112alsdud@naver.com)
- The lab scores will be uploaded to the LMS next week.
- Questions about Lab 3 scores will only be accepted during next lab session.

- code name: {student_id}_p3.c
- The code will be tested with 5 different input files.
- Each input file will be worth 20 points
- Make sure that the output format matches the provided example exactly.

Evaluation criteria

Category	Evaluation	
p3	100	
Total	100	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

Lab3 – Playlist Management Program Using Linked List

- You will implement a Playlist Management Program using a singly linked list.
- The input file contains commands followed by necessary arguments:
  - **i [songA] [songB]** → Inserts [songA] after [songB] in the playlist.
    If [songB] is 's', [songA] is inserted after header node.
  - **d [songA]** → Deletes [songA] from the playlist.
  - **f [songA]** → Finds the previous song of [songA] in the playlist.
  - **p** → Prints the entire playlist

```
i Beautiful s
i RESONANCE Beautiful
i TOUCH RESONANCE
i FactCheck Lemonade
d RESONANCE
d Candy
p
d Beautiful
p
f TOUCH
f Lemonade
d TOUCH
d TOUCH
p
```

Input

Program with singly linked list

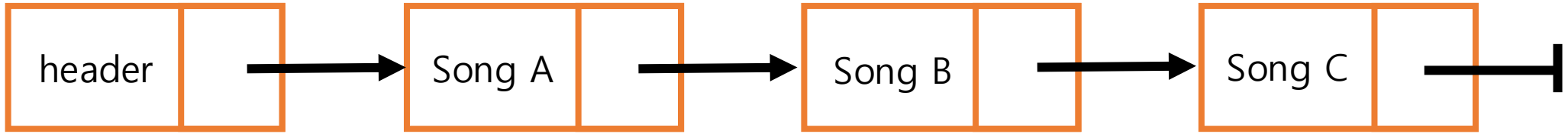
```
Insert Beautiful successfully!
Insert RESONANCE successfully!
Insert TOUCH successfully!
Insertion (FactCheck) Failed! Cannot find the location to be inserted.
Delete RESONANCE successfully!
Deletion Failed! Candy is not in the list.

[Playlist]
Beautiful -> TOUCH

Delete Beautiful successfully!

[Playlist]
TOUCH

TOUCH is the first song!
Error: Cannot find Lemonade.
Delete TOUCH successfully!
Deletion Failed! TOUCH is not in the list.
List is empty now!
```

Output

Lab3 – Playlist Management Program Using Linked List

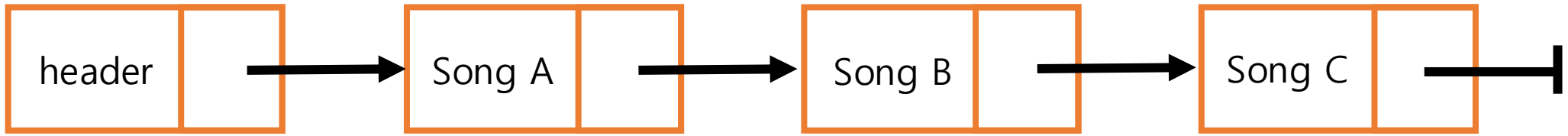
- **Your program should include:**
  - List MakeEmpty();
  - void Delete(ElementType X, List L);
  - Position FindPrevious(ElementType X, List L);
  - Position Find(ElementType X, List L);
  - void Insert(ElementType X, List L, Position P);
  - void DeleteList(List L);
  - void PrintList(List L);

Lab3 – Playlist Management Program Using Linked List



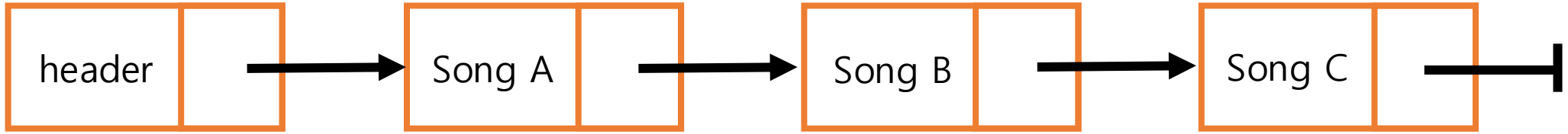
- **void MakeEmpty()**
  - Function to create an empty list with header node.
- **Position Find(ElementType X, List L)**
  - Function to find an element X in the list L.
- **Position FindPrevious(ElementType X, List L)**
  - Function to find the previous song of an element X in the list L.
  - Output message : **"Previous song of [song title]: [previous song title]"**
  - [Error handling] If the node cannot be found, please print an error message.
    "**Error: Cannot find [song title].**"
  - [Error handling] If it is the first node, please print an error message.
    "**[song title] is the first song!**"

Lab3 – Playlist Management Program Using Linked List



- **void Insert(ElementType X, List L, Position P)**
  - Function to insert a new element X in list L at a specific position P.
  - Output message : "**Insert [song title] successfully!**"
  - [Error handling] If the specified position is not found, please print an error message: "**Insertion ([song title]) Failed! Cannot find the location to be inserted.**"
- **void Delete(ElementType X, List L)**
  - Function to delete an element X from the list L.
  - Output message: "**Delete [song title] successfully!**"
  - [Error handling] If the song does not exist, please print an error message. "**Deletion Failed! [song title] is not in the list.**"

Lab3 – Playlist Management Program Using Linked List



- **void PrintList(List L)**
  - Function to print the playlist in a formatted way.
    "**[Playlist]**
    Song A -> Song B -> Song C"
  - [Error handling] If the list is empty, please print an error message.
    "**List is empty now!**"
- **void DeleteList(List L)**
  - Frees all memory allocated to the playlist.

Lab3 – Example

- Input file

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab3/code ❯ cat input1.txt
i Cake s
i BaggyJeans Cake
i Lucky s
i MyYouth Beginning
d Cake
i Perfume Lucky
d LittleLight
f Cake
f Lucky
f Perfume
p
```

- Result

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab3/code ❯ ./p3 input1.txt
Insert Cake successfully!
Insert BaggyJeans successfully!
Insert Lucky successfully!
Insertion (MyYouth) Failed! Cannot find the location to be inserted.
Delete Cake successfully!
Insert Perfume successfully!
Deletion Failed! LittleLight is not in the list.
Error: Cannot find Cake.
Lucky is the first song!
Previous song of Perfume: Lucky

[Playlist]
Lucky -> Perfume -> BaggyJeans
```

Lab3 – Sample code

- Structure

```
typedef struct Node* PtrToNode;
typedef PtrToNode List;
typedef PtrToNode Position;
typedef char ElementType[256];

struct Node {
        ElementType song;
        Position next;
};
```

- Function

```
List MakeEmpty();
void Delete(ElementType X, List L);
Position FindPrevious(ElementType X, List L);
Position Find(ElementType X, List L);
void Insert(ElementType X, List L, Position P);
void DeleteList(List L);
void PrintList(List L);
```

Lab3 – Sample code

```c
int main(int argc, char *argv[]){
        FILE *file = fopen(argv[1], "r");

        List playlist = MakeEmpty();
        char command;
        char song[256], prev_song[256];
        Position tmp;

        while (fscanf(file, " %c", &command) != EOF) {
                switch(command){
                        case 'i': //insert
                                fscanf(file, " %s %s", song, prev_song);
                                tmp = strcmp(prev_song, "s") == 0 ? playlist : Find(prev_song, playlist);
                                if (tmp == NULL) {
                                        printf("Insertion (%s) Failed! Cannot find the location to be inserted.\n", song);
                                }
                                else{
                                        Insert(song, playlist, tmp);
                                }
                                break;
                        case 'd': //delete
                                fscanf(file, " %s", song);
                                Delete(song, playlist);
                                break;
                        case 'p': //print
                                PrintList(playlist);
                                break;
                        case 'f':
                                fscanf(file, " %s", song);
                                tmp = FindPrevious(song, playlist);
                                if (tmp->next == NULL){
                                        printf("Error: Cannot find %s.\n", song);
                                }
                                else{
                                        if(strcmp(tmp->song, "\0") == 0){
                                                printf("%s is the first song!\n", song);
                                        }
                                        else{
                                                printf("Previous song of %s: %s\n", song, tmp->song);
                                        }
                                }
                                break;
                }
        }
        fclose(file);
        DeleteList(playlist);
        return 0;
}
```

```c
List MakeEmpty() {
        List L = (List)malloc(sizeof(struct Node));
        L->song[0] = '\0'; //empty header node!
        L->next = NULL;
        return L;
}
```